



Kestrel · Cheatsheet

Memory-safe, taint-aware systems language — brace syntax, LLVM backend.

v1.0.0 · quick reference

[kestrel-build.github.io](https://github.com/kestrel-build/kestrel)

01 CLI COMMANDS

<code>kestrel build <f></code>	Compile .kst to a native binary
<code>kestrel run <f></code>	Compile and run in one step
<code>kestrel check <f></code>	Type-check only, no binary
<code>kestrel new <n></code>	Scaffold a new project
<code>kestrel emit-ir <f></code>	Print Kestrel IR (debug)
<code>kestrel emit-llvm <f></code>	Emit LLVM IR (.ll)
<code>kestrel clean</code>	Remove build artifacts
<code>kestrel version</code>	Print version
<code>kestrel help</code>	Print help

02 CLI OPTIONS

<code>-o <out></code>	Output binary path
<code>--release</code>	Build with optimizations
<code>--verbose</code>	Print timing information
<code>--version</code>	Print version
<code>--help</code>	Print help

```
kestrel run hello.kst
kestrel build -o myapp --release main.kst
```

03 TYPES

<code>int32 int64</code>	Signed integers (fixed width)
<code>uint32 uint64</code>	Unsigned integers
<code>float32 float64</code>	IEEE 754 floats
<code>bool</code>	<code>true</code> / <code>false</code>
<code>str</code>	UTF-8 string (owned)
<code>T[N]</code>	Fixed-size array
<code>T[]</code>	Slice (borrowed view)
<code>List[T]</code>	Growable list
<code>HashMap[str,V]</code>	String-keyed hash map
<code>T?</code>	Nullable
<code>ptr[T]</code>	Raw pointer (unsafe)

```
int32 x = 42
float64 pi = 3.14159
str name = "Kestrel"
bool active = true
int32 flags = zeroed() // zero-init
auto n = 42             // inferred
const int32 MAX = 100
```

Mutable by default — use `const` for immutability. No `let` / `mut`.

04 FUNCTIONS

```
func add(int32 a, int32 b) -> int32 {
    return a + b
}

// -> void is required
func greet(str name) -> void {
    println("Hello, {name}!")
}

// generics use [T], not <T>
func identity[T](T val) -> T {
    return val
}
```

Interpolation: `"total = {a + b}"`. Booleans interpolate as `true` / `false`.

05 OPERATORS

Arithmetic	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code>
Comparison	<code>==</code> <code>!=</code> <code><</code> <code><=</code> <code>></code> <code>>=</code>
Logical	<code>&&</code> <code> </code> <code>!</code>
Bitwise	<code>&</code> <code> </code> <code>^</code> <code>~</code> <code><<</code> <code>>></code>
Assign	<code>=</code> <code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code> <code>%=</code>
	<code>&=</code> <code> =</code> <code>^=</code> <code><<=</code> <code>>>=</code>
Nullable	<code>?.</code> <code>??</code>

Logical is `&&` `||` `!` only — no `and` / `or` / `not`. `?.` safe call. `??` default.

06 CONTROL FLOW

```
if (x > 0) {
    // ...
} else if (x == 0) {
} else {
}

for (i in 0..10) { } // inclusive
for (i in 0..<10) { } // exclusive
for (item in list) { }
for (i, item in list) { }

while (cond) { }

match (color) {
    case Color.Red: handle_red()
    case Color.Blue: handle_blue()
    case _: handle_other()
}
```

One `for` loop — no `foreach`. Parens + braces required. Newline ends a statement (no semicolons).

07 COLLECTIONS

```
// List[T]
List[int32] xs = list_new()
xs.push(10)
int32 v = xs.get(0) // checked
int32 f = xs.first()
int32 l = xs.last()
int32 t = xs.pop() // rm+ret last
xs.sort() // ascending
xs.reverse()
bool h = xs.contains(10)
int64 at = xs.index_of(10) // or -1
bool e = xs.is_empty()
int64 n = xs.len()
xs.clear() // keep cap

// HashMap[str, V]
HashMap[str,int32] ages = map_new()
ages.insert("ada", 36)
int32 a = ages.get("ada")
bool k = ages.contains("ada")
```

Indexing is bounds-checked and aborts on out-of-range — no negative indexing; use `first()` / `last()`.

08 STRUCTS · CLASSES · ENUMS

```
// private by default; pub/protected
struct Point {
    float64 x
    float64 y
    pub func norm() -> float64 {
        return this.x*this.x + this.y*this.y
    }
}

// inheritance; 'parent', not 'base'
class Dog : Animal {
    override func speak() -> void {
        parent.speak()
        println("Woof!")
    }
}

// backing type required
enum Color : int32 {
    Red = 0
    Green = 1
    Blue = 2
}
Point p = Point { x: 1.0, y: 2.0 }
```

`clone()` is a deep copy — modifying a clone never touches the original.

09 ERROR HANDLING

```
try {
    File f = open("data.txt")
} catch (IOError e) {
    log(e.message)
} finally {
    cleanup()
}

defer close(resource) // scope cleanup
throw("something failed")
```

`try/catch/finally` only — there is no `?` operator.

10 CASTING & CONVERSIONS

```
float64 f = cast[float64](x) // safe
int32 n = f.to_int32() // method
const_cast[T](v) // drop const
unsafe_cast[T](v) // reinterpret
int64 sz = sizeof(Point)
```

11 NULLABLE TYPES

```
str? maybe = null
if (maybe != null) {
    println(maybe.len()) // narrowed
}
int32 len = maybe?.len() ?? 0
```

The compiler narrows `T?` to `T` after a `!= null` check.

12 CONCURRENCY

spawn

chan

atomic

mutex

rwlock

spinlock

semaphore

```
spawn worker() // green thread (Flight)
chan[int32] ch = chan()
atomic[int64] c = atomic(0)
c.fetch_add(1)
```

```
mutex[int32] m = mutex(0)
with (m.guard()) { // scoped lock
    // ... auto-unlocks
}
```

13 MEMORY — NOVA

Automatic, leak-free memory, **no GC**. Every value has an owner; heap buffers free at scope exit.

Declared

Owned

Moved

Borrowed

Locked

Freed

```
unsafe {
    ptr[int32] p = &x
    int32 val = *p
}
```

The six states above appear in diagnostics.

14 ANNOTATIONS

<code>@ignore_discard</code>	Callers may drop the return value (results are must-use by default)
<code>@no_return</code>	Function never returns
<code>@packed @aligned(N)</code>	Struct layout control
<code>@inline @cold @hot</code>	Optimization hints
<code>@section("n")</code>	Place symbol in a linker section
<code>@deprecated("m")</code>	Emit a deprecation warning
<code>@sensitive</code>	Encrypt value in the binary
<code>@tainted @sink</code>	Taint tracking (+@sanitizer)
<code>@test</code>	Mark a test function
<code>@repr("C")</code>	C-compatible layout

15 IMPORTS

```
import std.io.File
import std.collections.{HashMap, HashSet}
```

Use `import`, not `use`.